Tema 1 - Procesos

Tema 1 - Procesos
1.1.- Introducción y fundamentos

Programa: Serie de instrucciones que reciben datos y los transforman. Realizan una función específica de cara a un usuario.

Proceso: Programa en ejecución, en memoria. Elementos:

- Zonas de memoria donde se ponen los datos y las instrucciones.
- Conjunto de registros. Importantes para realizar los cambios de contexto. Destaca el registro contador.
- Estado del proceso. Lo veremos más adelante.
- Otra información: id del proceso, prioridades, estadísticas de uso.

Tema 1 - Procesos
1.1.- Introducción y fundamentos

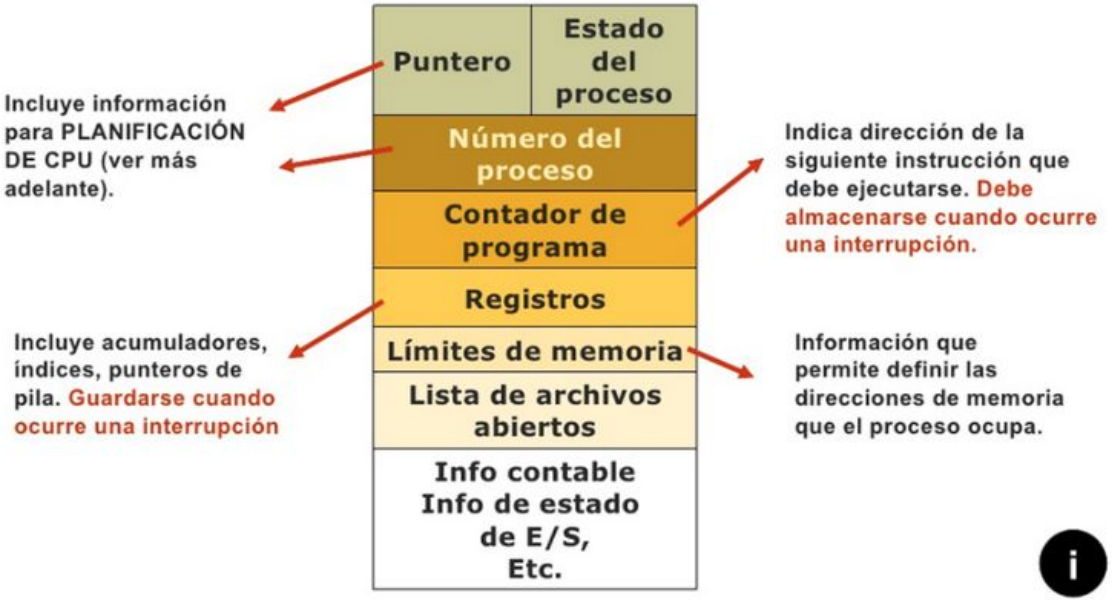
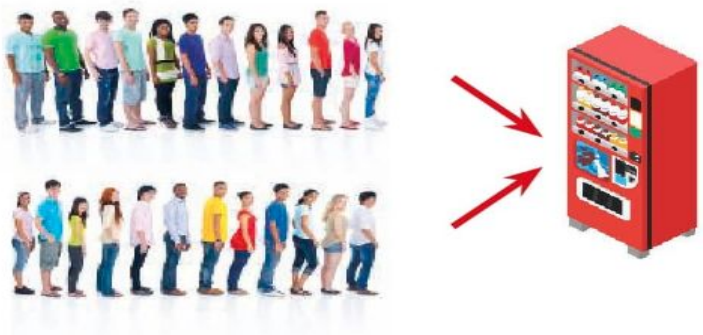


Imagen sacada de los apuntes del IES Doctor Balmis

Tema 1 - Procesos
1.2.- Concurrencia y distribución

- **Monoprogramación:** solo un proceso y hasta que no finaliza no entra otro. Se denomina también procesamiento por lotes (Batch).
- **Programación concurrente:** Varios procesos y esperan turno para ser atendidos.
    - Ejemplo - Varios hilos y reserva de billetes.



Tema 1 - Procesos
1.2.- Concurrencia y distribución

- **Quantum:** Intervalo de tiempo que el procesador destina a ejecutar un proceso. En cuanto termina pasa a ejecutar otro.

- **Cambio de contexto:** Guarda todo lo necesario del proceso para seguir después, y carga los parámetros necesarios para seguir con el nuevo.
  - ¿Qué ocurriría si el quantum es muy grande?
  - ¿Y si es muy pequeño?

- **Programación paralela:** Los procesos se ejecutan a la vez, en diferentes procesadores.

Tema 1 - Procesos
1.2.- Concurrencia y distribución

- **¿Concurrencia o paralelismo?**
  - Una sola puerta de entrada a un recinto cultural, pero manejamos varias colas con sus prioridades (embarazadas, parejas, etc.).
  - Tenemos varias colas y varias puertas.

- **Los procesos se particionan para que su ejecución sea lo mejor posible:**
  - **Manualmente:** El programador determina los hilos en los que se divide un proceso.
  - **Automáticamente (en desarrollo) :** Lo decide la máquina viendo que instrucciones son linealmente independientes.

- **Programación distribuida:** El programa se distribuye en diferentes procesos pero en diferentes máquinas. Para tareas con mucha exigencia. Ejemplos:
  - **Superordenadores**
  - Proyecto Seti
  - Cryptojacking

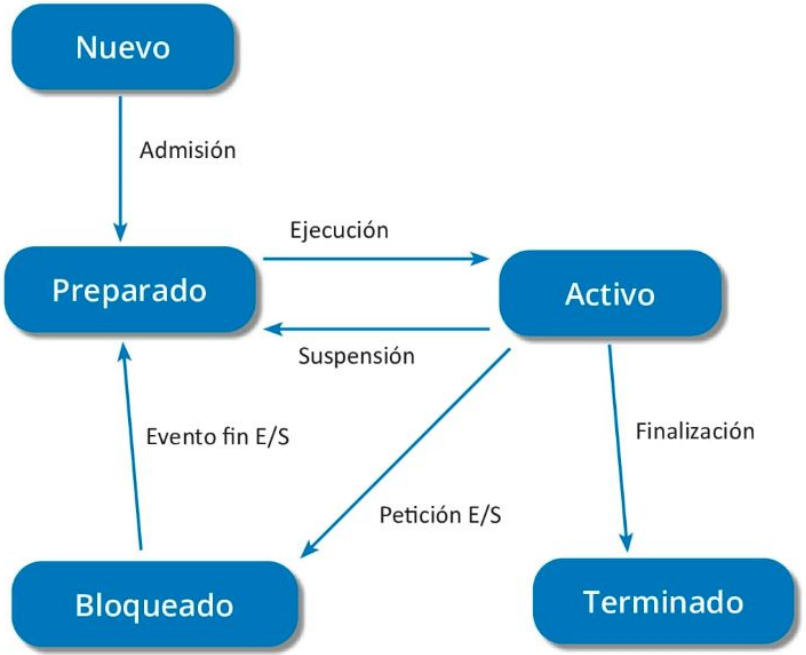
Tema 1 - Procesos
1.3.- Estados de un proceso.

- **PCB:** Bloque de control de procesos. Es una estructura que representa un proceso en memoria

A continuación, enumeramos los estados más significativos de un proceso:

- **Nuevo**. Proceso en creación.
- **Listo**. Proceso a la espera de ejecución de sus instrucciones por parte de la CPU.
- **Ejecución**. Proceso cuyas instrucciones están siendo ejecutadas por la CPU.
- **Terminado**. Proceso que ha finalizado.
- **Bloqueado**. Proceso a la espera de una operación de E/S.
- **Suspendido**. Proceso que se ha guardado en una memoria secundaria (disco duro, etc.) para liberar recursos en la memoria principal (RAM).

Tema 1 - Procesos
1.3.- Estados de un proceso.



Tema 1 - Procesos
1.4.- Gestión de procesos.

- Existe una tabla de procesos para que el sistema operativo maneje los PCB.
- Dos sistemas hardware básicos para realizar la gestión de procesos:
  - Mecanismo de **interrupciones**: Un proceso es interrumpido por el procesador o por otro proceso con mayor prioridad. Deben ser muy rápidas.
  - **modo usuario/kernel:** Existen instrucciones privilegiadas que solamente el kernel puede usarlas para realizar las interrupciones y los cambios de contexto, llamadas al sistema, etc....

  - Ejemplos: Movimientos del ratón, teclado, apagado.... Necesitan un mecanismo de interrupciones con una alta prioridad.

Tema 1 - Procesos
1.4- Gestión de procesos.

1.4.1-Dispatcher (Despachador).

- Encargado de ejecutar los cambios de contexto.
- Tiene lugar cuando:
  - Se agota el quantum.
  - Proceso termina.
  - Proceso inicia una operación de E/S.
  - De bloqueado a modo preparado.
  - Interrupción del sistema que saca al proceso.
- **Cambio de contexto:**
  - Guardar en qué instrucción se queda y la información que tenía en los registros.
  - Cargar la misma información pero con el proceso que entra.
  - Modificar el PCB.

Tema 1 - Procesos
1.4- Gestión de procesos.

1.4.1-PCB

- Identificador del proceso.
- Nombre.
- Estado.
- Registros de la CPU, en especial el PC (program counter).
- Prioridad.
- Punteros a las zonas de memoria que se manejan (instrucciones y datos).
- Propietario y privilegio del sistema de recursos.
- Estado de las operaciones de E/S.

Tema 1 - Procesos
1.4- Gestión de procesos.

1.4.2 .- Scheduler

- Encargado de planificar los procesos. Es decir, en base a un algoritmo específico y un posible sistema de prioridades, decidir qué proceso entra de la CPU.
- Algoritmos (con prioridad o no):
  - **FCFS** - Procesos se ejecutan en orden de llegada.
  - **SJF (Small Job First)** - Primero los que tengan menor tiempo de ejecución.
  - **SRTF** - SJF apropiativo. - Si llega uno con menos tiempo expulsa al que está en la CPU si ése tiene mayor tiempo restante.
  - **Round-Robin** - Se va rotando, usando el quantum, en la lista de preparados.
  - **Colas multinivel** - La idea es dividir a los procesos en **varias colas**, cada una **con su propio algoritmo de planificación y prioridad** y **no cambian de cola**.

Tema 1 - Procesos
1.4- Gestión de procesos.

1.4.2 .- Ejemplo de cola multinivel:

Imagina que el SO crea 3 colas:

1. **Cola de alta prioridad (tiempo real, interactivos):**
   - Procesos de usuario que requieren respuesta rápida (ejemplo: teclado, mouse, UI).
   - Se suele usar Round Robin con cuanto de tiempo corto.
2. **Cola de prioridad media (procesos de usuario normales):**
   - Aplicaciones comunes (navegador, editor de texto).
   - Se usa planificación por prioridad o Round Robin con cuanto más largo.
3. **Cola de prioridad baja (procesos en segundo plano / batch):**
   - Procesos largos o en lote (ejemplo: backup, impresión).
   - Se puede usar FCFS (First Come First Served).

Tema 1 - Procesos
1.5- Operaciones con procesos.

1.5.1 .- Fork

- Crea un proceso hijo. Solo funciona en sistemas unix/max.

```
# fork solo funciona en Unix/macOS
import os

def padre():
    while True:
        newpid = os.fork()
        if newpid == 0:
            hijo()
        else:
            pids = (os.getpid(), newpid)
            print(f"Padre: {os.getpid()}, Hijo: {newpid}")
            reply = input("Pulsa 's' si quieres crear un nuevo proceso: ")
            if reply != 's':
                break

def hijo():
    print('\n>>>>>>>>>>> Nuevo hijo creado con el pid %d a punto de finalizar <<<<<<' %, os.getpid())
    os._exit(0)

padre()
```

Tema 1 - Procesos
1.5- Operaciones con procesos.

1.5.1 .- Fork

- Crea el código y ejecútalo.
- Haz un pstree -p PID <padre>.
- ¿Siguen estando los procesos hijos ejecutándose?

Tema 1 - Procesos
1.5- Operaciones con procesos.

1.5.1 .- Fork

- **Servidor web.**
  - Un servidor (como **Apache en modo prefork**) crea un **proceso hijo por cada cliente** que se conecta:
  - El proceso **padre** está escuchando en un puerto (por ejemplo, el 80 para HTTP).
  - Cuando llega una conexión de un cliente, el padre hace un `fork()`.
  - El **proceso hijo** atiende a ese cliente (recibe su petición, envía la respuesta).
  - Mientras tanto, el **padre sigue vivo** y puede aceptar nuevas conexiones.
  - Cuando el hijo termina, se cierra.

```
import os
import socket

def servidor():
    # Crear un socket TCP
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind(("0.0.0.0", 12345))
    s.listen(5) #puede haber hasta 5 clientes esperando.
    print("Servidor escuchando en el puerto 12345...")

    while True:
        conn, addr = s.accept() #espera hasta que llega un cliente. conn-> socket para el cliente.
        print(f"Conexión desde {addr}")

        pid = os.fork()
        if pid == 0:
            # Proceso hijo: atiende al cliente
            s.close()  # El hijo no necesita el socket principal
            conn.sendall(b"Hola, cliente!\n")
            conn.close()
            os._exit(0)  # Hijo termina
        else:
            # Proceso padre: sigue aceptando conexiones
            conn.close()

servidor()
```

Tema 1 - Procesos
1.5- Operaciones con procesos (fork).

- **sleep 30 & ->** Cuando se manda un proceso a segundo plano, el padre crea con fork un proceso hijo que ejecuta el comando y así el padre queda libre para seguir ejecutando comandos.
  - El shell (padre) hace un fork().
  - El hijo ejecuta sleep 30 (se queda dormido 30 segundos).
  - El padre vuelve a mostrarte el prompt ($) y puedes seguir escribiendo comandos.

- **Ejercicio 1: Crear 3 procesos hijos usando os.fork().**
  - Cada hijo debe imprimir: "Soy el hijo X con PID Y" y luego terminar.
  - El padre debe imprimir: "Soy el padre con PID Z" después de crear cada hijo.
  - El programa debe asegurarse de que cada hijo termina antes de crear el siguiente.

    Para éllo debes usar el comando os.wait()
  - **Puntos clave a observar:**
    - Diferenciar padre e hijo (if fork() == 0)
    - Usar os._exit(0) para que el hijo termine correctamente

Tema 1 - Procesos
1.5- Operaciones con procesos (fork).

- **Ejercicio 2:**
  - Crear 5 procesos hijos usando os.fork().
  - **Cada hijo debe:**
    - Calcular la suma de los números del 1 al N, donde N = PID del hijo % 100 (para variar los números).
    - Imprimir su PID y el resultado de la suma.
    - Terminar correctamente (os._exit(0)).

  - **El padre debe:**
    - imprimir un mensaje cada vez que crea un hijo:
      "Creado hijo con PID X"
    - Esperar a que todos los hijos terminen. Para éllo:
      - Debes hacer un bucle, fuera del bucle de creación de los hijos, donde ejecutas un os.wait en cada interación.
    - Imprimir "Todos los hijos han terminado" al final.

Tema 1 - Procesos
1.5- Operaciones con procesos (fork).

- **Ejercicio 3:**
  - Escribe un programa que cree **5 procesos "trabajadores"**. Cada trabajador debe:

    i. Dormir un número aleatorio de segundos entre 1 y 5 (`random.randint(1, 5)`), simulando que hace una tarea.

  - El padre debe crear a los 5 sin esperar entre uno y otro, y luego ir informando **según van terminando** (no en el orden en que los creaste): algo como `"Ha terminado el trabajador X (PID NNNN)"`.

  - PIstas:
    i. Usa un bucle, fuera del bucle que crea los hijos, y utiliza `os.wait()` (sin argumento) que te devuelve el PID y el status del **primer** hijo que termine, sea cual sea.

    pid_terminado, stado = os.wait()

    ii. Vas a necesitar un diccionario, en java es HashMap, o Map. que asocie el PID con el número de trabajador y hacerlo en el momento en que creas cada hijo (en el padre, justo después del `fork`).

Tema 1 - Procesos
1.5- Operaciones con procesos (fork).

- **Ejercicio 3 - Esqueleto**

```
import os
import random

NUM_TRABAJADORES = 5
pid_a_num = {}

for num in range(NUM_TRABAJADORES):
    pid = os.fork()
    if pid == 0:
        # TODO: dormir un tiempo aleatorio y salir con os._exit(num)
        pass
    else:
        # TODO: guarda en pid_a_num la relación pid -> num
        pass

for _ in range(NUM_TRABAJADORES):
    pid_terminado, status = os.wait()
    # TODO: usa pid_a_num para saber qué trabajador era e imprime el aviso

print("Resumen: todos los trabajadores han terminado.")
```

Tema 1 - Procesos
1.5- Operaciones con procesos (fork).

- **Ejercicio 4:**

El padre define una lista con 5 nombres, por ejemplo `["Ana", "Luis", "Marta", "Diego", "Sara"]`, y hace `fork()`. El hijo debe recorrer la lista con un bucle `for` e imprimir cada nombre, uno por línea, con el formato `"Hijo (PID <pid>): <nombre>"`. El padre, mientras tanto, solo espera a que el hijo termine y luego imprime `"El padre ha terminado de esperar"`.

Pistas

- Recorrer una lista en Python se hace con `for elemento in lista:`.
- El hijo hereda una copia de la lista tal como estaba en el momento del `fork()`, así que puede usarla directamente.
- El hijo debe terminar con `os._exit(0)` después de imprimir, y el padre debe esperarlo con `os.waitpid(pid, 0)` (o `os.wait()`) antes de imprimir su propio mensaje.
- En la otra diapositiva tienes el esqueleto:

Tema 1 - Procesos
1.5- Operaciones con procesos (fork).

- **Ejercicio 4 - Esqueleto**

```
import os

nombres = ["Ana", "Luis", "Marta", "Diego", "Sara"]

pid = os.fork()

if pid == 0:

    # TODO: recorre "nombres" con un for e imprime cada uno

    # TODO: termina con os._exit(0)

    pass

else:

    # TODO: espera al hijo y luego imprime el mensaje del padre

    pass
```

Tema 1 - Procesos
1.5- Operaciones con procesos (fork).

- **Ejercicio 5**

El padre tiene una lista de 3 "archivos" simulados (cada uno es a su vez una lista de líneas de texto, como si fueran logs de un servidor). Crea 3 hijos, uno por archivo: cada hijo debe recorrer su lista de líneas y contar cuántas contienen la palabra `"ERROR"`, luego imprimir el resultado. El padre crea los 3 sin esperar entre ellos, y al final los espera a todos.

Pistas

- Recorre la lista archivos para ir sacando cada log de la forma `for archivo in archivos:`
- Buscar una palabra dentro de una línea se hace con el operador `in: if "ERROR" in linea` devuelve `True` o `False` directamente.
- Como cada hijo sólo necesita imprimir su resultado (no devolver nada al padre), no hace falta código de salida especial: basta con `os._exit(0)`.
- Esquema en la siguiente página:

- ## Ejercicio 5 - Esqueleto

```
import os

archivos = [
        ["INFO login ok", "ERROR timeout", "INFO logout ok"],
        ["ERROR db down", "ERROR db down", "INFO retry ok"],
        ["INFO ok", "INFO ok", "INFO ok"],
]

for archivo in archivos:
    pid = os.fork()
    if pid == 0:
        # TODO: cuenta cuántas líneas de "contenido" tienen "ERROR"
        # TODO: imprime algo como "Hijo 0 (PID ...): 3 líneas, 1 con ERROR"
        # TODO: termina con os._exit(0)
    # el padre no hace nada especial aquí, sigue creando el siguiente hijo
# TODO: espera a los 3 hijos (por ejemplo, os.wait() tres veces en un bucle)
print("El padre ha terminado de esperar a los 3 hijos")
```

Tema 1 - Procesos
1.5- Operaciones con procesos (fork).

- En windows se usa **Process**:

```
from multiprocessing import Process
import os

def hijo():
        print(f“Padre: {os.getppid()}, Hijo: {os.getpid()}”)
        os._exit(0)

def padre():
        while True:
                p = Process(target=hijo) -> se crea el procedimiento hijo apuntando a la función “hijo”.
                p.start()
                print (“\nNuevo hijo creado “ , p.pid)
                p.join() -> Espera a que el hijo termine para seguir el padre.
                opción = input(“Pulsa ‘s’ si quieres crear un nuevo proceso\n”)
                if opción != ‘s’:
                        break
padre()
```

Tema 1 - Procesos
1.5- Operaciones con procesos.
1.5.2.- Creación y ejecución

- Crear y manipular un proceso desde otro proceso.
- Creación simple de procesos:

```
import subprocess
try:
        resultado = subprocess.run(["ls", "-l"], text=True, capture_output=True)
        print("Salida:")
        print(resultado.stdout)
        print("Código de retorno:", resultado.returncode)

except subprocess.CalledProcessError as e:
        print(e.output)
```

Tema 1 - Procesos
1.5- Operaciones con procesos.
1.5.2.- Creación y ejecución

- ¿Qué devuelve subprocess.run?

```
import subprocess

# Ejecuta "echo Hola Mundo"
resultado = subprocess.run(["echo", "Hola Mundo"], capture_output=True, text=True)

print(resultado.args)      # ['echo', 'Hola Mundo']
print(resultado.returncode) # 0
print(resultado.stdout)    # "Hola Mundo\n"
print(resultado.stderr)    # None (no hubo error)
```

Tema 1 - Procesos
1.5- Operaciones con procesos.
1.5.2.- Creación y ejecución

- **Ejercicio:** Lanzar un proceso que abra el navegador con la búsqueda en Google de un término solicitado al usuario.

```
import subprocess
import urllib.parse

termino = input("Ingrese el término de búsqueda: ")
termino_codificado = urllib.parse.quote(termino)  # Codifica espacios y caracteres
url = f"https://www.google.com/search?q={termino_codificado}"

 # En Linux (xdg-open), MacOS (open) o Windows (start)
 try:
        subprocess.run(["xdg-open", url])  # Linux
 except FileNotFoundError:
        try:
                subprocess.run(["open", url])  # macOS
        except FileNotFoundError:
                subprocess.run(["start", url], shell=True)  # Windows

print("El navegador se ha abierto en un proceso hijo.")
```

Tema 1 - Procesos
1.5- Operaciones con procesos.
1.5.3.- Acceso

- **Popen** es igual que **run**, pero no espera a que el proceso lanzado acabe. run llama a popen que es de más bajo nivel.

```
import subprocess

# Lanzamos el comando ping
proceso = subprocess.Popen(
    ["ping", "-c", "5", "google.com"],  # -c 5 = hacer 5 pings (Linux/Mac)
    stdout=subprocess.PIPE, -> Envía la salida hacia python.
    stderr=subprocess.PIPE, -> Envía la salida de error hacia python.
    text=True #puede ser en modo texto o modo binario.
)

print("Haciendo ping en tiempo real...\n")

# Leemos línea por línea mientras el proceso sigue activo
numPing = 1
for linea in proceso.stdout:
    print(f"{numPing} >>", linea.strip())
    numPing+=1

print("\nPing terminado.")
```

Tema 1 - Procesos
1.5- Operaciones con procesos.
1.5.3.- Acceso

- **Popen** es igual que **run**, pero no espera a que el proceso lanzado acabe.
- **Ejercicio 1 con Popen:**
  - **Programa miPing.py que haga un ping -c 5 a google y que salga la siguiente salida, en vez de la estándar:**

```
El servidor 142.250.200.110 nos envía los datos : icmp_seq=1 ttl=119 time=21.4 ms
El servidor 142.250.200.110 nos envía los datos : icmp_seq=2 ttl=119 time=21.8 ms
El servidor 142.250.200.110 nos envía los datos : icmp_seq=3 ttl=119 time=21.8 ms
El servidor 142.250.200.110 nos envía los datos : icmp_seq=4 ttl=119 time=25.9 ms
El servidor 142.250.200.110 nos envía los datos : icmp_seq=5 ttl=119 time=51.5 ms
```

  - Debes investigar el comando strip(), comando split() y el comando find()
  - Tu profesor debe hablarte sobre las listas en python.
- **Ejercicio 2 con Popen:**
  - **Programa miMac.py que saca la macs de las tarjetas del equipo.**
  - La salida debe ser de la forma:
    - tarjeta1 -> mac1
    - tarjeta2 -> mac2
  - Debes usar el comando **ip a** con **Popen.**

Tema 1 - Procesos
1.5- Operaciones con procesos.
1.5.3.- Acceso

```
import subprocess

# Abrimos un proceso que convierte texto a mayúsculas con 'tr'
proceso = subprocess.Popen(
    ["tr", "a-z", "A-Z"], #tr se usa para intercambiar o eliminar caracteres desde archivo o stdin
    stdin=subprocess.PIPE, #poder escribir usando python en la entrada de buffer.
    stdout=subprocess.PIPE, #capturar la salida del programa y enviarla a python.
    text=True
)

# Escribimos varias veces
proceso.stdin.write("hola mundo\n")
proceso.stdin.write("python con popen\n")
proceso.stdin.flush() #Enviamos la salida hacia el proceso tr

# Cerramos stdin para indicar que no enviaremos más
proceso.stdin.close()

# Leemos la salida
salida = proceso.stdout.read()
print("=== SALIDA ===")
print(salida)
```

Tema 1 - Procesos
1.5- Operaciones con procesos.
1.5.3.- Acceso (Popen)

- **Ejercicio 1: Usando Popen y el comando tr:**
  - El usuario escribe texto en Python.
  - Se lo enviamos a `tr` para convertir minúsculas en mayúsculas.
  - Mostramos la respuesta **en tiempo real**.
  - Repetimos hasta que el usuario escriba `salir`.
- **Ejercicio 2: Encripta usando el método César y el comando tr.**
  - El usuario escribe texto en Python.
  - Se lo enviamos a `tr` para añadir un desplazamiento de 3. (usa chatgpt para saber el comando adecuado con tr).
  - Mostramos la respuesta **en tiempo real**.
  - Repetimos hasta que el usuario escriba `salir`.
- **Ejercicio 3, Ejercicio 4 y Ejercicio 5**: pincha en este enlace.

Tema 1 - Procesos
1.5- Operaciones con procesos.
1.5.4.- Propiedades.

- **psutil** (*Process and System Utilities*): **Obtener información del sistema y administrar procesos** de forma multiplataforma (funciona en Windows, macOS y Linux).
- Utilizada para tareas de **monitorización del sistema**, **gestión de recursos**, y **automatización administrativa**.
- Se debe instalar usando **apt install python3-psutil**.
- **Ejemplos de uso:**

1.- Información del sistema.

```
import psutil
print(psutil.cpu_percent(interval=1))      # Porcentaje de uso de CPU
print(psutil.cpu_count(logical=True))      # Núcleos lógicos
print(psutil.cpu_freq())                   # Frecuencia actual
print(psutil.virtual_memory())  # RAM total, usada, libre, porcentaje
print(psutil.swap_memory())     # Información de memoria swap
print(psutil.disk_usage('/'))   # Uso del disco
print(psutil.disk_partitions()) # Particiones montadas
print(psutil.net_io_counters()) # Bytes enviados/recibidos
print(psutil.net_if_addrs())    # Interfaces de red y direcciones IP
print(psutil.sensors_temperatures())
print(psutil.sensors_battery())
```

Tema 1 - Procesos
1.5- Operaciones con procesos.
1.5.4.- Propiedades.

2.- Gestión de procesos:

- **Listar procesos activos:**

```
for proc in psutil.process_iter(['pid', 'name']):
    print(proc.info)
```

- **Obtener información de un proceso específico:**

```
p = psutil.Process(1234)  # PID del proceso
print(p.name())
print(p.cpu_percent())
print(p.memory_info())
```

- **Matar un proceso específico:**

```
p.terminate()
```

- **Iniciar un nuevo proceso:**

```
p = psutil.Popen(["python3", "mi_script.py"])
```

1.5.5.- Eliminación y espera.

- **terminate()**: Termina un proceso, dejando que muera.
- **kill()**: Mata de forma tajante un proceso.
- Para saber que un proceso ya no existe, se usa la función **is_running().**

Tema 1 - Procesos
1.5- Operaciones con procesos.

1.5.5.- Eliminación y espera.

```
import psutil
import time

# Supongamos que conocemos el PID del proceso
pid = 1234  # reemplaza con el PID real

try:
    proceso = psutil.Process(pid)

    # TERMINATE: intenta cerrar el proceso de manera "suave"
    proceso.terminate()
    print(f"Se envió la señal terminate al proceso {pid}.")

    # Esperamos hasta 3 segundos a que termine
    try:
        proceso.wait(timeout=3) #se usa para esperar a que ese proceso termine.
        print(f"Proceso {pid} terminó correctamente con terminate.")
    except psutil.TimeoutExpired:
        print(f"Proceso {pid} no respondió a terminate, se usará kill.")
        # KILL: fuerza la terminación inmediata
        proceso.kill()
        proceso.wait(timeout=3)
        print(f"Proceso {pid} fue terminado con kill.")

    # POLL: verificamos si el proceso sigue activo
    if proceso.is_running():
        print(f"Proceso {pid} todavía está activo.")
    else:
        print(f"Proceso {pid} ya no está activo.")

except psutil.NoSuchProcess:
    print(f"No existe el proceso con PID {pid}.")
except psutil.AccessDenied:
    print(f"No tienes permisos para interactuar con el proceso {pid}.")
```

- **Ejercicio:** Programa que lista los procesos del sistema y mata uno determinado. Mostrará un menú:

Menú
1.- Listar procesos.
2.- Matar proceso.
3.- Salir.
Elige opción(1-3):